

Module 1

Verification Planning & Management Foundations (SV/UVM)

Learning objectives

- Learn how to turn a DUT specification into a concrete verification plan, with clear strategy, cover...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Verification Mindset and Goa"]

T2["From Specification to Verifi"]

T1 --> T2

T3["Verification Strategy (SV/UV)"]

T2 --> T3

Learn how to turn a DUT specification into a concrete verification plan, with clear strategy, coverage intent, and regression approach, usi...

Overview

- This module establishes the foundation for
 verification planning and management. Instead of
 sta...

How to learn this module

Suggested learning path (1/2)

- Read this module doc and ``module1/README.md`` for objectives and workspace layout.
- Scaffold planning files if needed:
``./scripts/module1.sh --scaffold``
- Edit ``module1/VERIFICATION_PLAN.md`` and ``module1/REQUIREMENTS_MATRIX.md`` from templates.
- Inspect shared DUT RTL at ``common_dut/rtl/stream_fifo.sv`` and the TB skeleton under ``common_dut/tb/...``
- Validate planning artifacts: ``./scripts/module1.sh --check``

Suggested learning path (2/2)

- Complete `module1/CHECKLIST.md` before starting Module 2.

Design architecture

1. Stream FIFO DUT (shared course RTL)

- Course DUT:
 - ``common_dut/rtl/stream_fifo.sv``
 - parameterizable ``DEPTH`` and ``DATA_WIDTH``.
- Source interface: ``s_valid``, ``s_ready``, ``s_data`` (push when both valid and ready).
- Sink interface: ``m_valid``, ``m_ready``, ``m_data`` (pop on handshake).
- Internal storage: circular buffer (``mem``, ``wr_ptr``, ``rd_ptr``, ``count``).
- Status outputs: ``level``, sticky

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
    subgraph DUT["stream_fifo (common_dut/rtl)"]
        SRC["Source interface\ns_valid / s_ready / s_data"]
        MEM["Circular buffer\nmem[DEPTH], wr_ptr, rd_ptr"]
        SNK["Sink interface\nm_valid / m_ready / m_data"]
        STS["Status\nlevel, overflow, underflow"]
    end
```

2. Planning-level verification view (Module 1)

- No full UVM implementation required yet — document intended env boundaries in
`VERIFICATION_PLAN.md...
- `REQUIREMENTS\_MATRIX.md` traces features → future tests and coverage bins.
- `common\_dut/tb/` holds skeleton stubs so architecture decisions are concrete, not abstract.

```
Verification Architecture
Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
SPEC["DUT spec"] --> PLAN["VERIFICATION_PLAN.md"]
PLAN --> MATRIX["REQUIREMENTS_MATRIX.md"]
MATRIX --> TB["common_dut/tb skeleton\n(env stubs)"]
PLAN --> CHECK["module1/CHECKLIST.md"]
```

3. Repository layout and planning workflow

- Inputs: DUT spec, course README, Module 1 templates and optional solutions.
- Workspace: edit ``module1/*.md`` planning documents directly in the repository.
- Shared RTL/TB: ``common_dut/`` holds the physical DUT and skeleton env reused in later modules.
- Validation loop: edit plan → run
``./scripts/module1.sh --check`` → fix TODOs → checklist sign-off.
- Outputs: verification scope, strategy, test catalogue intents, and coverage sketch for Module 2.

DUT — stream_fifo ports

```
module stream_fifo #(
    parameter int DATA_WIDTH = 8,
    parameter int DEPTH       = 16
) (
    input  logic                clk,
    input  logic                rst_n,

    // Source (input) interface
    input  logic                s_valid,
    input  logic [DATA_WIDTH-1:0] s_data,
    output logic                s_ready,

    // Sink (output) interface
    output logic                m_valid,
    output logic [DATA_WIDTH-1:0] m_data,
    input  logic                m_ready,

    # ...
```

UVM env skeleton

```
class stream_fifo_env_skeleton extends uvm_env;

    stream_fifo_agent_skeleton src_agent;
    stream_fifo_agent_skeleton snk_agent;

    `uvm_component_utils(stream_fifo_env_skeleton)

    function new(string name = "stream_fifo_env_skeleton", uvm_component
parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        src_agent =
stream_fifo_agent_skeleton::type_id::create("src_agent",
this);
        snk_agent
```

Key files to study

Open these in the repo (1/2)

- ``module1/VERIFICATION_PLAN.md`` — verification strategy, scope, and test intents
- ``module1/REQUIREMENTS_MATRIX.md`` — requirements-to-test traceability
- ``module1/templates/`` — empty templates for a fresh start
- ``module1/.solutions/`` — reference solutions (optional comparison)
- ``common_dut/rtl/stream_fifo.sv`` — shared course DUT (stream FIFO)
- ``common_dut/tb/stream_fifo_env_skeleton.sv`` — UVM env stub hierarchy

Open these in the repo (2/2)

- ``scripts/module1.sh`` — validation and scaffold orchestrator

Verification & testing methods

1. Planning-first methodology

- Start from spec → scope → strategy before writing large amounts of testbench code.
- Use risk-based prioritization to decide what to verify first in later modules.
- Separate verification objectives (quality/risk) from implementation details.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

REQ["Requirements intake"] --> SCOPE["Scope & risk prioritization"]

SCOPE --> STRAT["Strategy: directed / random / assertions"]

STRAT --> TESTS["Test intents catalogue"]

STRAT --> COV["Coverage intent sketch"]

TESTS --> VAL["./scripts/module1.sh --check"]

2. Test intent and validation (Module 1)

- Capture test intents (names, goals, tiers) in the verification plan — not full UVM tests yet.
- Directed vs constrained-random choices are documented with rationale for the FIFO DUT.
- Self-check: ``./scripts/module1.sh --check`` validates planning artifacts; simulation is optional.

3. Early closure thinking

- Sketch functional coverage intent and regression tiers (sanity / nightly / full) for later modules.
- Define sign-off preview criteria (coverage goals, bug bars) in the plan from day one.

Module 1 — validation commands

EXAMPLES:

```
# Full summary (plan, matrix, checklist)
```

```
$0
```

```
# Focus only on plan status
```

```
$0 --plan-status
```

```
# Focus only on traceability matrix
```

```
$0 --matrix-status
```

```
EOF
```

```
}
```

Syllabus topics

1. Verification Mindset and Goals (1/2)

- Design vs Verification
- What verification is responsible for (and what it is not).
- “Bug-finding” vs “quality and risk management”.
- Verification Objectives
- Proving conformance to spec (as far as practical).
- Managing risk: focusing on high-impact features and scenarios.

1. Verification Mindset and Goals (2/2)

- Balancing schedule, quality, and resources.
- UVM/SV Context
- Why constrained-random and coverage-driven verification (CDV) matter.
- Where SV/UVM and Verilog DUTs fit in the overall flow.

2. From Specification to Verification Scope (1/2)

- Requirements Intake
- Extracting verifiable requirements from a product/design spec.
- Identifying interfaces, modes, configurations, and corner cases.
- Capturing assumptions and dependencies (e.g., upstream/downstream blocks, firmware behavior).
- In-Scope vs Out-of-Scope
- Deciding what this verification effort will and will not cover.

2. From Specification to Verification Scope (2/2)

- Handling legacy behavior, debug features, and low-priority modes.
- Risk-Based Prioritization
- High-risk vs low-risk features.
- Safety/criticality, complexity, novelty, and usage frequency.

3. Verification Strategy (SV/UVM Focus) (1/2)

- Strategy Dimensions
- Directed vs constrained-random vs scenario-based testing.
- Use of reference models, assertions, and protocol checkers.
- Layered testbench and reuse (block/IP → subsystem → SoC).
- UVM Testbench Architecture Concepts
- How your env/agent/sequence/scoreboard choices influence planning.

3. Verification Strategy (SV/UVM Focus) (2/2)

- Mapping features and scenarios to sequences and configuration knobs.
- Choosing Strategies for the DUT
- For a FIFO: corner-case heavy, error injection, stress tests.
- For a bus/reg block: protocol correctness, access patterns, error responses.
- For a UART: timing, framing/parity errors, flow control.

3. Verification flow

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

SPEC["DUT spec"] --> PLAN["VERIFICATION_PLAN.md"]

PLAN --> MATRIX["REQUIREMENTS_MATRIX.md"]

MATRIX --> TB["common_dut/tb skeleton\n(env stubs)"]

PLAN --> CHECK["module1/CHECKLIST.md"]

Stimulus, observation, and check strategy for this module.

4. Test Planning and Test Catalogue Basics (1/2)

- Test Intent vs Test Implementation
- Writing test descriptions at the intent level (what to prove).
- Mapping intents to one or more concrete UVM tests/sequences.
- Test Types and Tiers
- Smoke/sanity vs feature tests vs stress/error-injection.
- Nightly vs weekly regressions (preview; detailed in later modules).

4. Test Planning and Test Catalogue Basics (2/2)

- Early Test Catalogue
- Building a first-pass list of 10–20 key tests/intents from the spec.
- Tagging each with requirements, priority, and expected coverage impact.

5. Coverage Planning Primer (1/2)

- Why Coverage is Central to Planning
- Functional vs code coverage and their roles.
- “Coverage as a question”: what do you need to know to sleep at night?
- Sketching Coverage Models
- Listing features, modes, and scenarios that deserve covergroups.
- Identifying important crosses (e.g., mode × error type, address ranges × operations).

5. Coverage Planning Primer (2/2)

- Defining Initial Coverage Goals
- Setting measurable but realistic targets.
- Separating “must cover” from “nice to have”.

6. Intro to Regression and Sign-off Thinking

- Regression at a High Level
- Why we need repeatable, automated regressions.
- Basic tiers (sanity / nightly / full).
- Sign-off Preview
- Types of sign-off criteria: coverage, bug bars, waivers, stability.
- Why it's important to think about exit criteria from Day 1.

Command reference highlights

Scaffold planning workspace

- `./scripts/module1.sh --scaffold`

Validate Module 1 artifacts

- `./scripts/module1.sh --check`

Inspect templates and shared DUT

- `head -30 module1/templates/VERIFICATION_PLAN.md`

Hands-on examples

Module 1 self-check

```
$ ./scripts/module1.sh --help
```

Terminal: module1_check

```
[[0;36m=====[[0m
[[0;36mModule 1: Verification Planning & Management Foundations[[0m
[[0;36m=====[[0m
```

Usage: ./scripts/module1.sh [OPTIONS]

Module 1: Verification Planning & Management Foundations (SV/UVM)
This script summarizes and checks the planning artifacts for Module 1.

OPTIONS:

```
--plan-status      Show status of verification_plan.md only
--matrix-status    Show status of requirements_matrix.md only
--checklist-status Show status of checklist_module1.md only
--structure-status Show required-sections check only
--traceability-status Show Req ID traceability check only
--summary          Show all statuses (default)
--check            Media/CI self-check (structure only)
--demo             Print example commands from EXAMPLES.md
--scaffold         Copy templates/*.md into module1/ if missing
--help, -h        Show this help message
```

EXAMPLES:

```
# Full summary (plan, matrix, checklist)
./scripts/module1.sh

# Focus only on plan status
./scripts/module1.sh --plan-status

# Focus only on traceability matrix
./scripts/module1.sh --matrix-status
```

Exercise scaffold

```
./scripts/module1.sh --scaffold
```

Demo: Verification plan template

```
$ head -30 module1/templates/VERIFICATION_PLAN.md
```

Terminal: ex01_plan_template

Verification Plan Template (Module 1)

> **Purpose**: Capture the initial verification plan for your chosen DUT \

> **Module**: 1 – Verification Planning & Management Foundations (SV/UVM)

> **Instructions**: Fill in this document for your design. Copy from `module1/templates/` if nee

1. Project and DUT Overview

> **How to fill**: Add project name, DUT name, owners, version. Describe the DUT in 2-4 sentence

<!-- TODO: Project name, DUT name, owners, version. DUT description, interfaces, operating modes

2. Verification Objectives and Scope

> **How to fill**: List 3-5 verification goals (e.g. prove correctness, meet coverage, limit esc

<!-- TODO: Objectives, in-scope, out-of-scope, risk assessment. -->

3. Verification Strategy (SV/UVM)

> **How to fill**: Describe directed vs constrained-random vs scenario-based approach. Outline p

<!-- TODO: Approach (directed vs random), env architecture, checkers/assertions strategy. -->

4. Test Planning and Test Catalogue

> **How to fill**: Define test tiers (smoke, feature, stress, error). List an initial catalogue

<!-- TODO: Test taxonomy, initial catalogue (>=10), mapping to requirements. -->

Demo: Requirements matrix template

```
$ head -25 module1/templates/REQUIREMENTS_MATRIX.md
```

Terminal: ex02_matrix_template

```
# Requirements-Verification Traceability Matrix (Module 1)
```

```
> Purpose: Track mapping from requirements → tests → coverage items → checks \
```

```
> Module: 1 – Verification Planning & Management Foundations (SV/UVM)
```

```
> Instructions: Fill in this document for your design. Copy from `module1/templates/` if nee
```

```
## 1. Instructions
```

```
> How to fill: Add 1-2 sentences on how you will maintain requirement IDs (e.g. R1, R2, ...) a
```

```
<!-- TODO: Add notes on how you'll maintain IDs and mappings. -->
```

```
## 2. Requirement List
```

```
> How to fill: Add a markdown table with columns: Req ID, Description, Priority (H/M/L), Not
```

```
<!-- TODO: Table of Req ID, description, priority, notes. -->
```

```
## 3. Requirement → Test Mapping
```

```
> How to fill: For each requirement, list which test IDs exercise it. Use a table or bullet
```

```
<!-- TODO: Map each requirement to one or more tests (Test IDs). -->
```

Demo: Reference solution

```
$ head -20 module1/.solutions/VERIFICATION_PLAN.md
```

Terminal: ex03_solutions

Verification Plan Template (Module 1)

> **Purpose**: Capture the initial verification plan for your chosen DUT
> **Module**: 1 – Verification Planning & Management Foundations (SV/UVM)

1. Project and DUT Overview

- **Project name**: Stream FIFO Verification
- **Block/DUT name**: `stream\_fifo`
- **Owner(s)**: Yongfu
- **Date / version**: 2026-02-03 / v0.1

1.1 DUT Description

- **High-level function**:
 - `stream\_fifo` is a parameterizable streaming FIFO that buffers data between a source and a sink.
 - It stores up to `DEPTH` elements of `DATA\_WIDTH` bits each and guarantees in-order delivery.
 - It exposes status information (current fill level) and sticky overflow/underflow flags to the user.
- **External interfaces** (ports, buses, clocks, resets):
 - `clk`, `rst\_n`: single clock and active-low asynchronous reset for all internal state.

Demo: Shared DUT RTL

```
$ head -35 common_dut/rtl/stream_fifo.sv
```

Terminal: ex04_dut_rtl

```
`timescale 1ns/1ps

// Simple parameterizable streaming FIFO with ready/valid handshakes.
//
// This module is intended as a common DUT for the verification planning
// exercises. It exposes:
// - An input (source) interface: s_valid/s_ready/s_data
// - An output (sink) interface: m_valid/m_ready/m_data
// - Status: current fill level, sticky overflow/underflow flags
//
// Behavior:
// - When s_valid && s_ready, s_data is written into the FIFO.
// - s_ready is high when there is at least one free entry in the FIFO.
// - When m_valid && m_ready, the oldest entry is popped.
// - m_valid is high when there is at least one entry in the FIFO.
// - If s_valid is high while FIFO is full (s_ready == 0), overflow is set.
// - If m_ready is high while FIFO is empty (m_valid == 0), underflow is set.
// - overflow/underflow are sticky until reset.
//
module stream_fifo #(
    parameter int DATA_WIDTH = 8,
    parameter int DEPTH      = 16
) (
    input logic      clk,
    input logic      rst_n,

    // Source (input) interface
    input logic      s_valid,
    input logic [DATA_WIDTH-1:0] s_data,
    output logic      s_ready,

    // Sink (output) interface
    output logic      m_valid,
    output logic [DATA_WIDTH-1:0] m_data,
    input logic      m_ready,
```

Demo: Module validation

```
$ ./scripts/module1.sh --check
```

Terminal: ex05_validate

```
███[0;36m=====███[0m
███[0;36mModule 1: Verification Planning & Management Foundations███[0m
███[0;36m=====███[0m
```

Module 1: media self-check
OK: module1/ exists
OK: templates/ exists
OK: EXAMPLES.md exists
OK: docs/MODULE1.md exists
OK: common_dut/rtl/ exists

All required checks passed.

Practice & assessment

What you should know (1/6)

- Explain the role of verification and its relationship to design.
- Turn a DUT specification into a verification scope with clear assumptions and boundaries.
- Draft a verification strategy for a simple Verilog DUT and SV/UVM environment.
- Create an initial test catalogue (test intents) linked to requirements.
- Sketch a coverage plan (what to measure and rough coverage goals).
- Describe a basic regression concept and outline high-level sign-off ideas.

What you should know (2/6)

- Given a short DUT spec (e.g., FIFO or UART), list:
- Functional requirements (what the DUT must do).
- Non-functional/quality requirements (throughput, latency constraints, etc.).
- Assumptions and environment constraints.
- Classify each requirement as must-have, nice-to-have, or future.
- Mark items explicitly as in-scope or out-of-scope for this project.

What you should know (3/6)

- Choose your DUT (FIFO / UART / AXI-lite block / your own).
- In `module1/VERIFICATION\_PLAN.md`, add sections for:
- Objectives and success criteria.
- High-level SV/UVM environment concept (env, agents, scoreboard, reference model).
- Use of directed, random, and scenario-based tests.
- Planned use of assertions and protocol checkers.

What you should know (4/6)

- Keep the strategy to 1–2 pages maximum.
- Create a table with columns:
- Test ID
- Test name
- Intent / description
- Related requirement IDs

What you should know (5/6)

- Priority (H/M/L)
- Type (smoke/feature/stress/error)
- Populate at least 10 test intents for your DUT.
- From your requirement list and test catalogue, identify:
 - Signals/fields that deserve coverpoints (e.g., addresses, opcodes, status flags, error types).
- Crosses that matter (e.g., mode × error_type, burst_length × address_range).

What you should know (6/6)

- Write a brief description (no code required yet) of:
- Where each covergroup would live in the UVM testbench (monitor, scoreboard, etc.).
- What hitting those bins would tell you about verification completeness.

Exercises

- Scope and Risk Matrix
- Assumption Log
- UVM Architecture Sketch

Assessment checklist

- Can explain verification's role and goals in the SV/UVM context.
- Has produced a written verification scope (in-scope, out-of-scope, assumptions).
- Has a concise verification strategy document for the chosen DUT.
- Has an initial test catalogue with traceability to requirements.
- Has a high-level coverage plan (what to cover and why).
- Can articulate a basic regression and sign-off concept, even if details come later.

Summary & next steps

- Learn how to turn a DUT specification into a concrete verification plan, with clear strategy, cover...
- Complete module1/CHECKLIST.md
- Review module1/EXAMPLES.md and run each lab
- Next: Test Planning & Strategy in Depth